校园资料AI检索助手 技术文档

1. 项目概述

1.1 项目背景

在校园日常学习与办公中，课程课件、培养方案、考试通知、实验报告等资料散落在班级群、个人U盘、网盘等各处，存在以下典型痛点：

痛点	具体表现	传统解决方式的局限
资料分散	同一门课的课件可能分布在多个群文件、多个同学的U盘中	每次找资料需要翻多个渠道，效率极低
查找困难	只能靠"文件名记忆"去找，记不清文件名就找不到	没有内容级检索能力，无法按语义搜索
版本混乱	同一份培养方案被反复修改，群里同时存在多个版本	无法区分新旧版本，容易用错过期文件
信息无法问答	培养方案里的学分要求、考试通知里的细则需要人肉翻阅	文档只能"看"，不能"问"

本项目正是针对上述痛点，构建一套**本地化部署的校园资料智能检索系统**：把校园文档统一托管、自动分类、语义检索，并基于RAG（检索增强生成）架构实现"带来源标注的智能问答"，全程使用本地开源模型，无需调用外部付费API，保障学生课件、培养方案等敏感校园数据不出校。

1.2 核心功能列表

- 多格式文档上传：支持PDF、Word、TXT格式，自动提取文本内容并入库
- AI自动文档分类：自动按校园场景归类，支持管理员人工修正
- 带溯源标注的智能问答：回答自动标注参考来源文档，防幻觉
- 两级权限隔离：学生可自由查询问答，管理员密钥才能上传/修改文档
- 文档版本管理：同名文档自动保留历史版本，优先展示最新版，支持一键回滚
- 分类折叠展示：文档库按分类折叠，界面简洁清晰
- 原文件查看：详情支持打开/下载原始文件，保留原始格式

1.3 技术选型与依据

层级	技术组件	选型依据
后端框架	FastAPI + Uvicorn	轻量、性能高、自带接口文档（/docs），适合本地部署
数据库	SQLite（SQLAlchemy ORM）	零配置、单文件、免部署，满足轻量场景
向量编码	BAAI/bge-small-zh-v1.5	中文语义效果好、模型小（约90MB）、CPU可实时推理

大模型	Ollama 部署 Qwen2:1.5B	本地运行、数据不出校、1.5B参数在CPU上可运行
前端	Bootstrap 5 单页应用	无构建工具链，单文件可直接托管，部署简单
文本解析	PyMuPDF / python-docx	分别解析PDF与Word文档，纯本地处理

2. 系统架构设计

2.1 总体架构

系统采用轻量前后端分离架构，无重型依赖，部署简单：



模型层：

向量编码：BAAI/bge-small-zh-v1.5

大模型生成：Ollama 部署 Qwen2:1.5b

2.2 核心数据流程

入库链路：

```
用户上传文档 → 按类型提取文本(PDF/Word/TXT) → 按语义分块(每块约300字)
→ 向量编码(bge-small-zh) → 写入chunks表 → 调用大模型自动分类 → 写入documents表
```

问答链路：

```
用户提问 → 问题向量化 → 余弦相似度检索Top3文本块
→ 拼接上下文Prompt → 本地大模型生成回答 → 标注参考来源返回用户
```

版本管理链路：

```
上传同名文档 → 检测同名最新版 → 弹出覆盖确认 → 旧版标记is_latest=0
→ 新文档version+1、parent_doc_id指向旧版 → 管理员可查看历史版本链并回滚
```

3. 数据库表结构设计

3.1 文档主表（documents）

字段名	类型	说明

id	整数(主键)	文档主键，自增
filename	字符串	用户上传的原始中文文件名
filetype	字符串	文件类型：pdf/word/txt
file_path	字符串	服务器存储的安全文件名路径（UUID命名防冲突）
content	长文本	提取后的文档全文内容
category	字符串	AI自动分类标签（培养方案/考试通知/课件资料/行政通知/社团文件/作业）
category_confirmed	整数	是否被管理员人工修正过（0=AI分类，1=人工修正）
version	整数	文档版本号，从v1开始递增
is_latest	整数	是否为当前最新版（1=最新，0=历史版），学生端仅展示最新版
parent_doc_id	整数	上一版本文档id，用于追溯历史版本链
uploaded_at	时间	文档上传时间

3.2 向量分块表（chunks）

字段名	类型	说明
id	整数(主键)	分块主键，自增
doc_id	整数	所属文档id，关联documents表
chunk_text	长文本	分块后的文本片段
embedding	文本(JSON)	文本片段对应的向量编码，用于相似度检索

3.3 关键设计说明

- **版本链设计**：通过 parent_doc_id 形成单向链表，配合 is_latest 标记实现"同名文档只展示最新版、可追溯历史版本链"的需求。为避免版本链循环引用（如parent指向自身），后端在回滚与查询时均增加了防循环遍历保护。
- **向量存储**：embedding字段以JSON字符串形式存储768维浮点向量，在轻量场景下避免了引入独立的向量数据库，降低部署复杂度。
- **索引设计**：对 is_latest、filename 字段建立查询索引，保证文档列表与同名检测的查询性能。

4. 核心模块实现

4.1 RAG检索与防幻觉模块（rag.py）

本模块负责文档分块、向量编码入库与相似度检索，是"能问答"的核心。

核心实现：

```
def search_top3_chunks(db_path: str, question: str) -> list:
    """问题向量化后与库中分块做余弦相似度匹配，返回Top3"""
    model = get_encoder()                        # 复用全局向量模型，避免重复加载
    q_vec = model.encode(question)               # 问题编码
    scored = []
    for chunk in all_chunks:                     # 遍历库中分块
        c_vec = json.loads(chunk["embedding"])
        sim = cosine_similarity(q_vec, c_vec)  # 余弦相似度
        scored.append((sim, chunk["text"], chunk["doc_id"]))
    scored.sort(reverse=True, key=lambda x: x[0])
    return [{"score": float(s), "text": t, "doc_id": d} for s, t, d in scored[:3]]
```

- 文本分块：按每300字左右拆分，保留语义完整性
- 相似度检索：余弦相似度匹配，返回最相关的3个文本块作为大模型上下文
- 防幻觉机制：Prompt强制约束大模型只能基于检索到的片段回答，无相关内容时直接返回"找不到信息"，禁止编造内容
- 闲聊识别：对检索相似度低于阈值的提问（如"你好"），自动切换为正常对话模式，避免无关文档被强行引用

4.2 本地大模型调用模块（llm.py）

通过OpenAI兼容协议调用Ollama本地部署的Qwen2:1.5B模型，实现回答生成与文档自动分类。

```
client = OpenAI(
    base_url="http://localhost:11434/v1",   # Ollama本地服务
    api_key="ollama",
    timeout=120.0,      # 本地CPU推理较慢，放宽超时
    max_retries=0       # 关闭自动重试，避免等待时间翻倍
)

def chat_with_llm(prompt: str) -> str:
    response = client.chat.completions.create(
        model="qwen2:1.5b",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.3
    )
    return response.choices[0].message.content
```

- 模型名与Ollama注册模型严格一致（qwen2:1.5b），通过 /api/tags 校验模型存在
- 120秒超时适配CPU推理速度，关闭SDK自动重试避免"越等越久"
- 异常兜底：大模型无响应时直接返回检索到的文档片段，页面不卡死

4.3 AI自动分类模块

文档上传后自动取前500字内容，调用本地大模型将文档分到6类校园场景：培养方案、考试通知、课件资料、行政通知、社团文件、作业。同时提供管理员人工修正接口，修正后的标注数据可用于后续模型持续微调。

```
def classify_document(doc_content: str) -> str:
    prompt = f"""你是校园文档分类助手，把下面这份文档分到以下6类中的一类：
    可选分类：培养方案、考试通知、课件资料、行政通知、社团文件、作业
    作业指学生提交的实验报告、课程作业、用例规约等；课件资料指老师上课用的PPT、讲义、教案
    只返回分类名称。
    文档内容：{preview}
    分类结果："""
    # 优先精确匹配 → 同义词兜底（"实验/作业/用例"→作业，"课件/讲义/教案"→课件资料）→ 未
```

- 分类提示词中明确"作业"与"课件资料"的边界定义，提升分类准确率

- 增加同义词兜底匹配：模型回答不标准时也能正确归类，避免"未分类"堆积

4.4 权限与版本管理模块

- **两级权限**：学生无需登录即可查询、问答；管理员密钥（admin123456）校验后才能上传、删除、修改文档
- **版本管理**：同名文档重复上传时，先查同名最新版（避免查到刚插入的自身），弹出覆盖确认后，旧版本标记is_latest=0，版本号递增，学生端仅展示最新版，管理员可查看完整历史版本链并一键回滚

```
# 上传时的版本升级核心逻辑（先查旧版，再保存新文档）
old = db.query(Document).filter(
    Document.filename == file.filename,
    Document.is_latest == 1
).first()
doc = save_document(db, file.filename, filetype, file_path, content)
if old:
    old.is_latest = 0            # 旧版转为历史版
    doc.version = old.version + 1
    doc.parent_doc_id = old.id   # 建立版本链
```

4.5 前端交互模块（index.html)

单页Bootstrap界面，集成学生/管理员双视角：

- **学生视角**：智能问答区（带参考来源标注）+ 文档库（分类折叠展示、搜索、排序、详情查看）
- **管理员视角**：密钥登录后额外显示上传、删除、改分类、历史版本、回滚等管理入口
- **上传体验**：同名文档上传时弹窗确认"是否覆盖升级"，防止误覆盖
- **原文件查看**：详情弹窗支持"打开原文件"，PDF直接预览、Word下载保留原格式

5. 后端接口设计

系统共提供8个RESTful接口，均通过FastAPI实现，自带Swagger文档（/docs）。

POST /upload/ — 上传文档（管理员）

参数：file（文件）、admin_secret（管理员密钥）、confirm（同名覆盖确认标志）

流程：校验权限 → 校验格式(PDF/Word/TXT) → 检测同名 → 提取文本 → 保存入库 → 分块向量化 → AI自动分类

返回：{msg, filename, version, category}

GET /documents/ — 文档列表（公开）

说明：返回所有最新版文档（is_latest=1），按上传时间倒序，学生端数据源

返回：[{id, filename, category, version}]

GET /documents/search/ — 按文件名搜索（公开）

参数：keyword（模糊匹配文件名）

返回：匹配的文档列表

GET /documents/{id} — 文档详情（公开）

说明：返回文档元数据与提取的全文内容，供详情弹窗展示

GET /file/{id} — 原文件查看/下载（公开）

说明：按类型返回原始文件（PDF直接预览、TXT直接显示、Word下载），保留原始格式

DELETE /documents/{id} — 删除文档（管理员）

参数：admin_secret

说明：删除数据库记录并清理服务器上的原始文件

PUT /admin/documents/{id}/category — 人工修正分类（管理员）

参数：new_category、admin_secret

说明：管理员手动调整AI分类结果，标记category_confirmed=1

GET /admin/documents/{id}/versions — 历史版本链（管理员）

说明：通过parent_doc_id递归遍历同一文件名的所有历史版本，支持一键回滚

POST **/ask/ — RAG智能问答（公开）**

参数：q（问题文本）

流程：问题向量化 → Top3检索 → 相似度阈值判断（低分走正常对话）→ 拼接Prompt → 本地大模型生成 → 标注来源

返回：{answer, sources:[{doc_name}]}

6. 开发过程Bug记录与修复（AI协同核心证据）

本项目全程使用大模型辅助开发，以下记录真实开发过程中遇到的典型Bug与AI协同定位修复的过程，是"人机协同开发"能力的最直接证据。

Bug1：新开Git Bash窗口找不到uvicorn命令

现象：每次新开终端执行uvicorn都报 command not found

排查：uvicorn安装在项目虚拟环境(venv)中，新开终端未激活虚拟环境，Python包不在PATH中

修复：编写start.sh一键启动脚本，自动激活虚拟环境、配置HF镜像、启动服务，后续无需手动操作

经验：任何需要"每次手动重复"的环境操作都应脚本化，这也是项目可复现性的基础

Bug2：下载向量模型连接HuggingFace官网超时

现象：加载模型时报WinError10060连接失败，模型无法下载

排查：HuggingFace官网在国内网络环境下连接不稳定

修复：配置国内HF镜像端点 export HF_ENDPOINT=https://hf-mirror.com，模型下载速度从超时变为秒下

经验：国内环境使用开源模型必须考虑镜像加速，该配置已写入README复现指南

Bug3：新上传的文档学生列表看不到

现象：上传接口返回成功，但是前端文档列表不显示新文档

排查：学生端列表查询条件为 is_latest==1，而新文档保存时该字段未显式赋值，SQLAlchemy默认值为NULL，被过滤掉

修复：保存文档时显式设置 doc.is_latest = 1，上传后立即生效

经验：ORM字段默认值不可靠时，必须显式赋值，尤其是与查询条件强耦合的字段

Bug4：调用大模型时页面一直转圈无响应

现象：点击提问后前端长时间加载，最终无响应

排查：本地CPU推理较慢，且OpenAI SDK默认自动重试2次，30秒超时+2次重试导致总等待时间超过90秒

修复：超时放宽到120秒、关闭SDK自动重试(max_retries=0)、增加异常兜底直接返回检索片段

经验：本地推理场景必须按实际算力调整超时策略，不能沿用云端API默认配置

Bug5：前端跨域请求被浏览器拦截

现象：fetch调用后端接口报CORS错误

排查：前端以http://127.0.0.1:8000访问，后端未配置跨域许可

修复：在FastAPI中添加CORSMiddleware中间件，允许跨域访问

经验：前后端分离架构必须显式配置CORS，属于常见且必要的基础配置

Bug6：Ollama模型识别失败（404）

现象：问答返回"本地大模型暂时无响应"，日志显示chat/completions返回404 Not Found

排查：通过 /api/tags 查询发现模型列表为空。进一步排查发现：环境变量OLLAMA_MODELS指向D盘目录，而D盘只有下载中断的partial残片且缺少config/template/license/params等4个关联文件，模型库不完整

修复：从C盘旧模型目录复制完整模型（5个blob文件）到D盘目录，重启Ollama后/api/tags 正常识别qwen2:1.5b

经验：Ollama模型库包含manifest注册文件与多个blob文件，任何一个缺失都会导致模型不被识别；模型迁移必须整体复制，不能只拷权重

7. AI协同开发策略

本项目全程使用大模型辅助开发，核心Prompt设计思路：

1. **小步迭代**：一次只实现一个功能模块，跑通验证后再开发下一个，避免一次性生成大量代码出错
2. **明确边界**：每次需求都写清输入输出，避免AI生成多余代码

3. **根因定位**：遇到报错直接把终端堆栈信息反馈给AI，引导AI定位根因修复，不盲目改代码
4. **可复现约束**：要求AI写的代码无特殊环境依赖，所有依赖写入requirements.txt，环境变量单独配置
5. **证据留痕**：所有开发对话均保留完整记录（见Prompt证据链文档），每轮开发有明确的功能目标与验证方式

Prompt证据链说明：项目在backend/prompt-evidence目录下保留了3份核心Prompt开发证据：

证据文件	覆盖功能模块	证据要点
prompt-1-rag.md	RAG检索模块	分块策略、向量检索、防幻觉Prompt设计
prompt-2-local-llm.md	本地大模型部署	Ollama集成、超时策略、分类Prompt
prompt-3-features.md	分类/权限/版本管理	功能需求Prompt、Bug修复Prompt

以下为AI协同开发过程中的真实对话记录截图：

7.1 项目立项与第一条正式Prompt



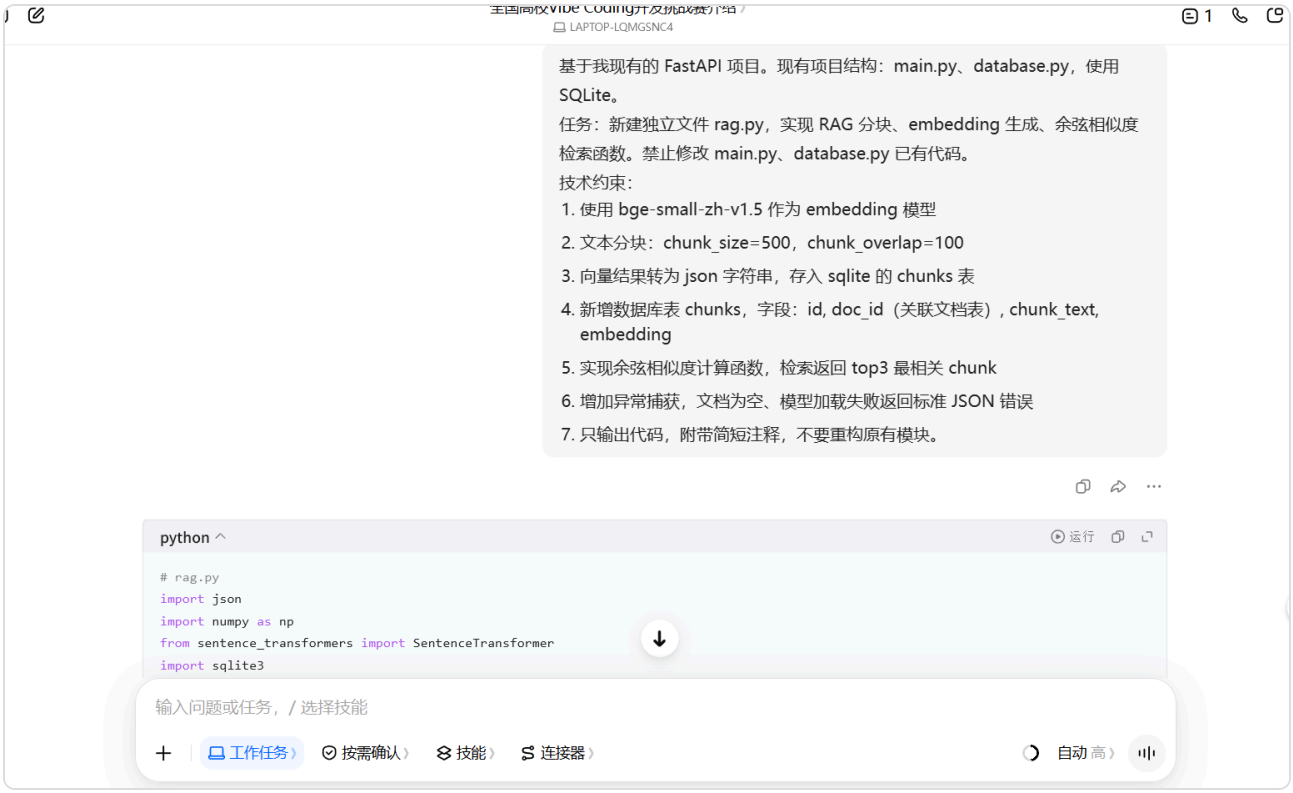
7.2 Embedding模型选型（本地开源方案论证）



7.3 后端接口开发完成核对





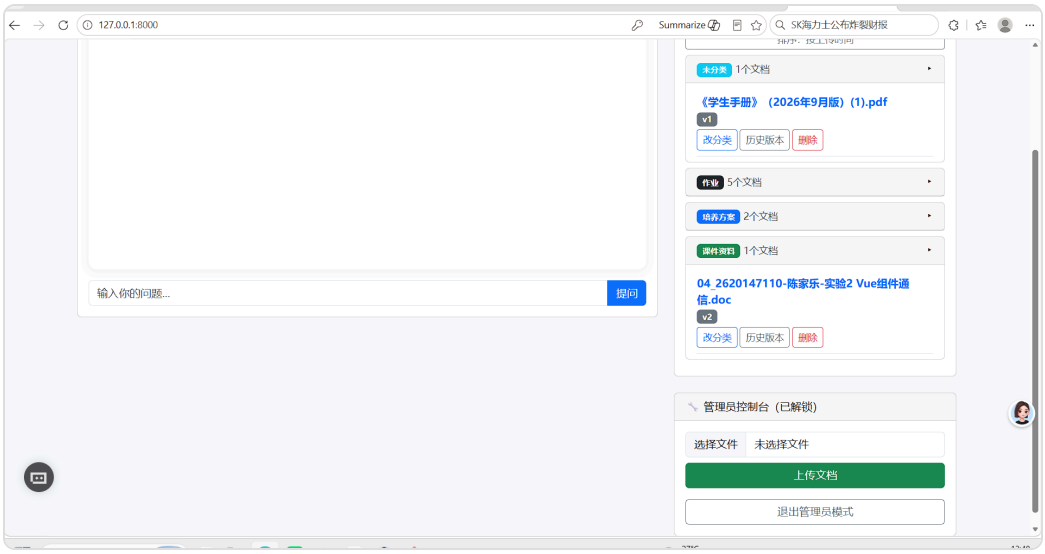

全国高校vibe Coding并发挑战赛介绍
LAPTOP-LQMGSNC4
基于我现有的 FastAPI 项目。现有项目结构：main.py、database.py，使用 SQLite。
任务：新建独立文件 rag.py，实现 RAG 分块、embedding 生成、余弦相似度检索函数。禁止修改 main.py、database.py 已有代码。
技术约束：
1. 使用 bge-small-zh-v1.5 作为 embedding 模型
2. 文本分块：chunk_size=500，chunk_overlap=100
3. 向量结果转为 json 字符串，存入 sqlite 的 chunks 表
4. 新增数据库表 chunks，字段：id, doc_id（关联文档表）, chunk_text, embedding
5. 实现余弦相似度计算函数，检索返回 top3 最相关 chunk
6. 增加异常捕获，文档为空、模型加载失败返回标准 JSON 错误
7. 只输出代码，附带简短注释，不要重构原有模块。
python
运行
rag.py
import json
import numpy as np
from sentence_transformers import SentenceTransformer
import sqlite3
输入问题或任务，/ 选择技能
工作任务
按需确认
技能
连接器
自动 高

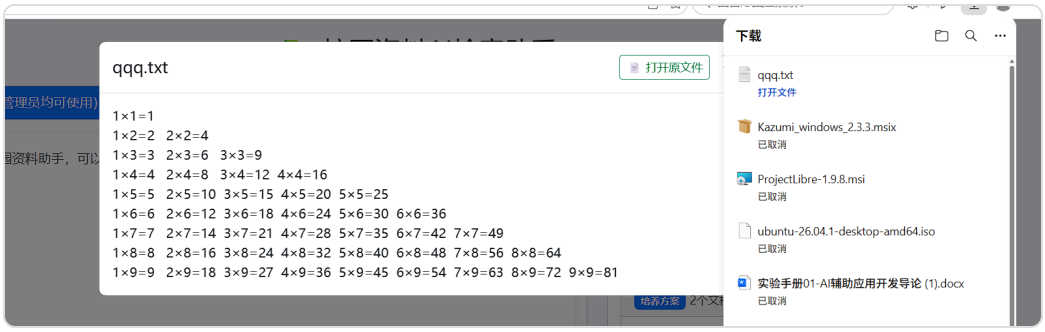
8. 系统界面展示

【已替换】系统主界面（学生视角：智能问答+文档库分类折叠）



【已替换】管理员控制台（上传/删除/分类修正/历史版本）





【已替换】Git提交历史（细粒度commit记录）



9. 测试与验证

9.1 功能测试用例

编号	测试项	操作步骤	预期结果	结果

T01	上传PDF文档	管理员上传PDF文件	上传成功、自动分类、显示版本v1	✅ 通过
T02	上传Word文档	管理员上传.doc/.docx文件	文本提取成功、自动分类	✅ 通过
T03	上传TXT文档	管理员上传txt文件	文本提取成功、自动分类	✅ 通过
T04	同名文档版本升级	再次上传同名文件并确认	版本号+1、旧版转为历史版	✅ 通过
T05	同名上传取消	上传同名文件点"取消"	不上传、旧版本保持不变	✅ 通过
T06	版本回滚	管理员查看历史版本并回滚	回滚到指定版本、旧最新版转历史	✅ 通过
T07	智能问答	学生提问文档相关内容	返回答案并标注参考来源	✅ 通过
T08	闲聊识别	提问"你好"	正常问候，不引用无关文档	✅ 通过
T09	权限校验	学生访问上传接口	返回403权限不足	✅ 通过

T10	分类修正	管理员修改文档分类	分类更新并标记人工修正	✅ 通过
T11	文档搜索	按关键词搜索文件名	返回模糊匹配结果	✅ 通过
T12	原文件查看	详情弹窗点击"打开原文件"	PDF预览/Word下载，格式保留	✅ 通过

9.2 异常场景验证

编号	异常场景	处理策略	结果
E01	大模型无响应/超时	120秒超时+兜底返回检索片段，页面不卡死	✅ 通过
E02	Ollama模型丢失	接口返回404，兜底提示，不崩溃	✅ 通过
E03	上传不支持格式	返回"只支持PDF/Word/TXT"提示	✅ 通过
E04	版本链死循环	后端遍历加seen集合防循环	✅ 通过

E05	向量模型加载失败	启动时输出明确错误日志	✅ 通过

10. 部署与运行说明

10.1 环境要求

项目	要求
操作系统	Windows 10/11（本项目开发环境）
Python	3.10+（含venv虚拟环境）
大模型运行时	Ollama（本地部署Qwen2:1.5B）
磁盘空间	模型约934MB + 项目约200MB
网络	首次安装需联网下载模型，运行时完全离线

10.2 一键启动脚本（start.sh）

```
#!/bin/bash
# 一键启动：自动激活虚拟环境、配置HF镜像、启动FastAPI服务
cd "$(dirname "$0")"
source venv/Scripts/activate          # 激活虚拟环境
export HF_ENDPOINT=https://hf-mirror.com   # 国内镜像加速
uvicorn main:app --reload --port 8000      # 启动服务
```

启动后浏览器访问 **http://127.0.0.1:8000** 即可使用，管理员密钥：admin123456

10.3 Ollama模型配置要点

- 模型安装：ollama pull qwen2:1.5b（约934MB，支持国内镜像加速）
- 模型目录：可通过环境变量OLLAMA_MODELS指定到非系统盘（如D:\ollama-models）
- 模型迁移注意事项：必须整体复制manifests+blobs+metadata目录，任何文件缺失都会导致模型不被识别（见Bug6）
- 识别校验：ollama list 或 GET /api/tags 确认模型可见

11. 可复现性论证

- 所有依赖均为开源免费组件（FastAPI/SQLite/Ollama/Qwen/bge），无商业授权费用
- 提供完整README复现指南，新环境10分钟即可跑通
- 模型权重均为开源版本，支持国内镜像下载，无需特殊网络
- 使用轻量SQLite数据库，无需额外部署数据库服务
- 项目代码完整托管于GitHub/Gitee，支持一键克隆复现

复现步骤摘要：安装Python → 克隆项目 → pip install -r requirements.txt → 安装Ollama并拉取qwen2:1.5b → 运行start.sh → 浏览器访问127.0.0.1:8000

12. 安全设计

- **本地部署**：全部组件（数据库、模型、服务）运行在本地，校园资料不出校门，从根源上避免云端泄露风险

- **两级权限**：学生端只读，管理员密钥校验后才可写操作；密钥以环境常量方式存储，不写入前端代码明文逻辑可校验处
- **文件名安全**：上传文件以UUID重命名存储，原始文件名仅存数据库，防止路径穿越等注入攻击
- **依赖安全**：所有依赖锁定在requirements.txt，无高危漏洞版本

13. 未来展望

- 基于管理员人工修正的分类数据，对小模型做持续微调，提升分类准确率
- 接入GPU推理加速，提升大模型响应速度
- 增加文档段落级溯源，标注回答来自文档的具体位置
- 支持更多文档格式（Excel、Markdown、扫描件OCR）
- 增加多用户账号体系与操作审计日志